



21. Swapping Mechanisms

Beyond Physical Memory: Mechanisms

Swap Space

Present Bit

Concept

Page fault

Page Replacement

Page Fault Control Flow

Hardware

Software

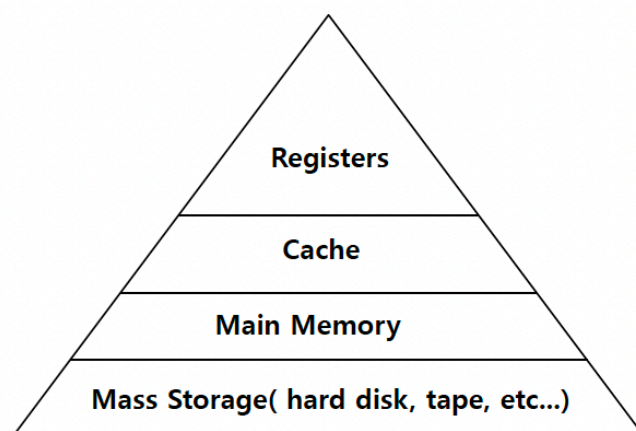
When to perform replacement

Lazy approach

Swap Daemon, Page Daemon

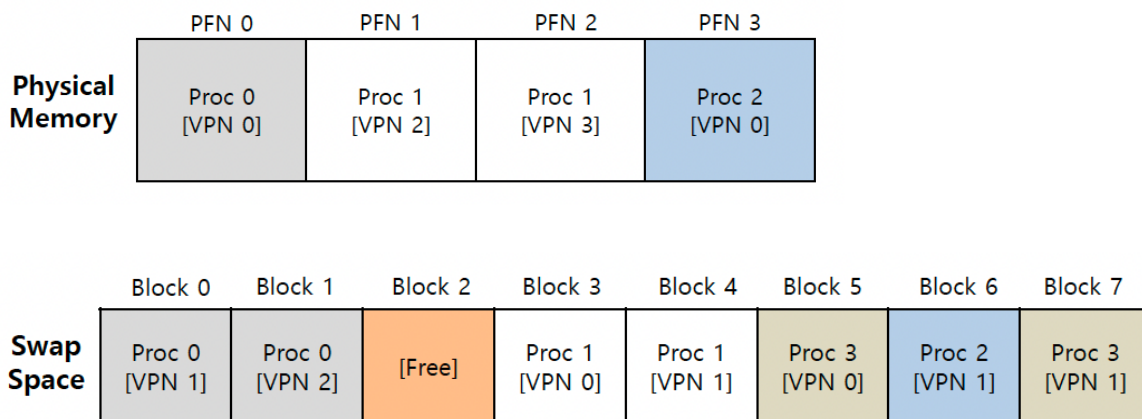
Beyond Physical Memory: Mechanisms

- 하드디스크의 일부를 마치 메인 메모리처럼 사용할 수 있도록 하는 기술
 - OS need a place to stash away portions of address space that currently aren't in great demand.
 - In modern systems, this role is usually served by a hard disk drive.



Swap Space

- Reserve some space on the disk for moving pages back and forth
먼저, 페이지를 이동할 수 있도록 **디스크의 공간을 확보**해야 한다
- OS needs to remember the swap space, in **page-sized unit**
- 회색이 process 0번



Present Bit

- Add some machinery higher up in the system in order to support swapping the pages to and from the disk.
 - When the hardware looks in the PTE, it may find that the page is not present in physical memory
PTE(Page Table Entry)를 봤는데 **present bit가 0**이면? → 그 페이지가 지금 물리 메모리에 없다는 뜻이야
이러면 **page fault**가 발생

```
// 1단계: VPN 추출
VPN = (VirtualAddress & VPN_MASK) >> SHIFT
(Success, TlbEntry) = TLB_Lookup(VPN)
// TLB Hit
// 권한 체크 => 물리 주소 계산 => 메모리 접근
if(Success == True)
    if(CanAccess(TlbEntry.ProtectBits) == True)
        Offset = VirtualAddress & OFFSET_MASK
```

```

    PhysAddr = (TlbEntry.PFN << SHIFT) | Offset
    Register = AccessMemory(PhysAddr)
else RaiseException(PROTECTION_FAULT)
else // TLB Miss
    // Page Table 확인
    PTEAddr = PTBR + (VPN * sizeof(PTE))
    PTE = AccessMemory(PTEAddr)
    if (PTE.Valid == False)
        // 잘못된 주소?
        RaiseException(SEGMENTATION_FAULT)
    else
        // 권한 없음
        if (CanAccess(PTE.ProtectBits) == False)
            RaiseException(PROTECTION_FAULT)
        // TLB에 넣고 재시도 (메모리에 있음)
        else if (PTE.Present == True)
            // assuming hardware-managed TLB
            TLB_Insert(VPN, PTE.PFN, PTE.ProtectBits)
            RetryInstruction()
        // 메모리에 없어 PAGE_FAULT 발생
        else if (PTE.Present == False)
            RaiseException(PAGE_FAULT)

```

Value	Meaning
1	page is present in physical memory
0	The page is not in memory but rather on disk

Concept

Page fault

- Accessing page that is **not in physical memory**
- If a page is not present and has been swapped disk, the OS needs to swap the page back into memory in order to service the page fault

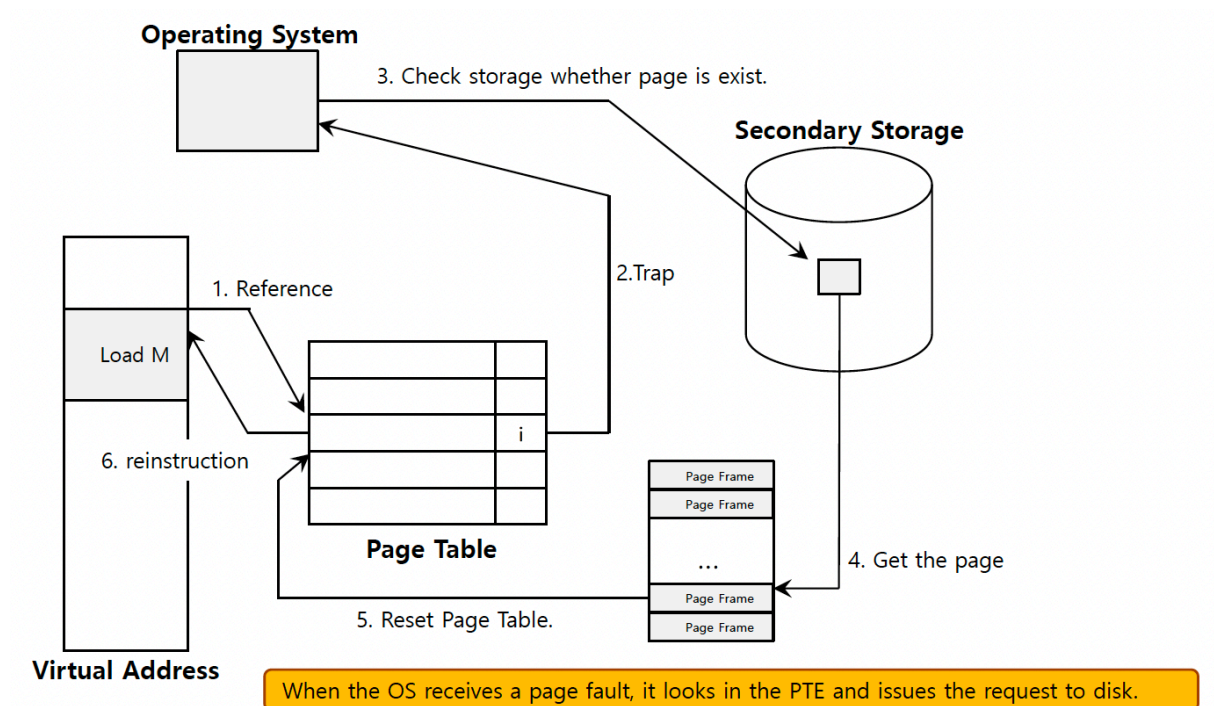
Page Replacement

- 메모리를 모두 사용 중인 상태 혹은 조금의 여유공간만 존재하는 상태에서 새로운 프로세스를 실행하고 싶다면 어떻게 할까?
- The process of picking a page to kick out or replace is known as **page-replacement** policy.

Page Fault Control Flow

- PTE used for data such as the PFN of the page for a disk address.

Hardware



Software

- 첫 번째 코드에서 `PTE.Present == False` 일 때 실행되는 거임

```
// 빈 물리 페이지 프레임 찾기
PFN = FindFreePhysicalPage()
// 메모리 짝 참
if (PFN == -1) // no free page found
```

```
PFN = EvictPage() // 교체 알고리즘 (LRU 등)
// 공통: 디스크에서 읽어오기
DiskRead(PTE.DiskAddr, pfn) // sleep (I/O 대기)
PTE.present = True // update page table with present
PTE.PFN = PFN // bit and translation (PFN)
RetryInstruction() // retry instruction
```

- The OS must find a physical frame for the soon-be-faulted-in page to reside within.
- If there is no such page, waiting for the replacement algorithm to run and kick some pages out of memory.

When to perform replacement

Lazy approach

- 메모리가 **완전히 꽉 찰 때까지** 기다렸다가, 그제서야 페이지를 쫓아내서 공간을 확보하는 방식이야.
- 비현실적

Swap Daemon, Page Daemon

- 가용 페이지 수가 **LW(Low Watermark)** 밑으로 떨어지면
→ 백그라운드 스레드(responsible for freeing memory)가 깨어남
- 이 스레드가 페이지들을 evict해서 **HW(High Watermark)**까지 가용 페이지를 확보

처음에 VA로 TLB에서 찾아보고 없으면 PTE로 가는거?

reinstruction

page fault control flow에서 tlb까지 생각해라